

ProductionManagementAI — Identity スキーマ — テーブル定義書

000_DB — 認証／RBAC 基盤（work-items/WI-001/brief.md INFRA-002）を支え、0002_ADR（docs/en/architecture/0002_ADR_auth-rbac-foundation.md）を実装する。

物理名の命名規則：ai/skills/database-design/SKILL.md に従い snake_case（「規則が確定するまでは snake_case の物理名を一貫して使う」）。ASP.NET Core Identity の既定の PascalCase のテーブル名（AspNetUsers など）は、AppDbContext の EF Core Fluent API の ToTable(...) 呼び出しで snake_case に付け替える。列名は EFCore.NamingConventions（work-items/WI-001/plan.md の認証基盤の設計で提案）で一貫して snake_case にする。

テーブル一覧

テーブル	物理名	目的
Users	users	アプリのユーザーアカウント（Identity の IdentityUser<Guid> + アプリ固有の項目）
Roles	roles	RBAC のロール（Identity の IdentityRole<Guid>）。Admin、Operator を初期登録
UserRoles	user_roles	ユーザーとロールの多対多の結合
UserClaims	user_claims	Identity の既定 — ユーザーごとのクレーム（まだどの機能も使っていないが、拡張性のため残す）
RoleClaims	role_claims	Identity の既定 — ロールごとのクレーム（まだどの機能も使っていないが、拡張性のため残す）
UserLogins	user_logins	Identity の既定 — 外部ログインプロバイダー（未使用。外部ログインは対象外）
UserTokens	user_tokens	Identity の既定 — ユーザーごとのトークン、例：パスワードリセット（まだどの機能も使っていないが、拡張性のため残す）

ER 図とリレーション

テーブル	関連テーブル	リレーション (1:1 / 1:N / N:N)	FK 列
users	roles	N:N（user_roles 経由）	user_roles.user_id / user_roles.role_id
users	user_claims	1:N	user_claims.user_id
users	user_logins	1:N	user_logins.user_id
users	user_tokens	1:N	user_tokens.user_id

テーブル	関連テーブル	リレーション（1:1 / 1:N / N:N）	FK 列
roles	role_claims	1:N	role_claims.role_id

テーブル定義

users

項目名	物理名	データ型	長さ	NOT NULL	既定値	PK/FK	備考
Id	id	uuid	—	yes	gen_random_uuid()	PK	Identity の IdentityUser<Guid> のキー
UserName	user_name	varchar	256	no	なし	—	Identity の既定
NormalizedUserName	normalized_user_name	varchar	256	no	なし	—	大文字・小文字を区別しない検索のため大文字化。一意インデックス
Email	email	varchar	256	no	なし	—	Identity の既定
NormalizedEmail	normalized_email	varchar	256	no	なし	—	大文字・小文字を区別しない検索のため大文字化
EmailConfirmed	email_confirmed	boolean	—	yes	false	—	Identity の既定。まだどの機能も使わない（メール確認のフローは対象外）
PasswordHash	password_hash	text	—	no	なし	—	Identity が管理するハッシュ。Identity の API 以外では読み書きしない
SecurityStamp	security_stamp	text	—	no	なし	—	Identity の既定。資格情報の変更で無効化される
ConcurrencyStamp	concurrency_stamp	text	—	no	なし	—	Identity の既定。楽観的同時実行制御
PhoneNumber	phone_number	varchar	32	no	なし	—	Identity の既定。未使用
PhoneNumberConfirmed	phone_number_confirmed	boolean	—	yes	false	—	Identity の既定。未使用
TwoFactorEnabled	two_factor_enabled	boolean	—	yes	false	—	Identity の既定。2 要素認証のフローは対象外
LockoutEnd	lockout_end	timestampz	—	no	なし	—	Identity の既定。0002_ADR の STRIDE のロックアウト対策を支える
LockoutEnabled	lockout_enabled	boolean	—	yes	true	—	Identity の既定
AccessFailedCount	access_failed_count	integer	—	yes	0	—	Identity の既定
DisplayName	display_name	varchar	200	yes	なし	—	アプリ固有の追加
IsActive	is_active	boolean	—	yes	true	—	アプリ固有の追加。将来のアカウント無効化のユースケースのために確保。まだどの要件も使わない

項目名	物理名	データ型	長さ	NOT NULL	既定値	PK/FK	備考
CreatedAtUtc	created_at_utc	timestampz	—	yes	now()	—	アプリ固有の追加

roles

項目名	物理名	データ型	長さ	NOT NULL	既定値	PK/FK	備考
Id	id	uuid	—	yes	gen_random_uuid()	PK	Identity の IdentityRole<Guid> のキー
Name	name	varchar	256	no	なし	—	Admin / Operator、初期登録
NormalizedName	normalized_name	varchar	256	no	なし	—	大文字化。一意インデックス
ConcurrencyStamp	concurrency_stamp	text	—	no	なし	—	Identity の既定

user_roles

項目名	物理名	データ型	長さ	NOT NULL	既定値	PK/FK	備考
UserId	user_id	uuid	—	yes	なし	PK、FK → users.id	role_id との複合 PK
RoleId	role_id	uuid	—	yes	なし	PK、FK → roles.id	user_id との複合 PK

user_claims、role_claims、user_logins、user_tokens

ASP.NET Core Identity 標準の EF Core の列の対応のまま残す（work-items/WI-001/plan.md の認証基盤の設計のセクションのとおり、残すコストはほぼゼロ：「後で外部ログインやクレームが必要になったときの互換性のないスキーマ変更を避ける」）。どの列も Identity の既定の形から外れないため、ここでは列ごとに挙げない。IdentityDbContext<AppUser, AppRole, Guid, ...> が自動生成し、マイグレーション生成時にレビューする（work-items/WI-001/plan.md ステップ 11）。

インデックス定義

インデックス名	テーブル	列	種類	根拠（アクセスパターン）
ix_users_normalized_user_name	users	normalized_user_name	一意 btree	ログインは正規化したユーザー名で検索する。Identity は一意性を必要とする
ix_users_normalized_email	users	normalized_email	btree	将来のメールアドレスでの検索を支える。一意ではない（Identity の既定の RequireUniqueEmail は明示的に有効にしない限りオフ — brief.md はメールアドレスでのログインを必要としないためオフのまま）

インデックス名	テーブル	列	種類	根拠（アクセスパターン）
ix_roles_normalized_name	roles	normalized_name	一意 btree	名前によるロールの検索と初期登録の冪等性
pk_user_roles	user_roles	user_id, role_id	一意 btree (PK)	主な検索経路：「このユーザーはどのロールを持つか」

制約

制約	種類 (PK/FK/UNIQUE/CHECK)	テーブル.列	参照先	ルール
pk_users	PK	users.id	—	—
pk_roles	PK	roles.id	—	—
pk_user_roles	PK	user_roles.(user_id, role_id)	—	—
fk_user_roles_user_id	FK	user_roles.user_id	users.id	ON DELETE CASCADE — ユーザーを削除するとそのロールの割り当ても削除される
fk_user_roles_role_id	FK	user_roles.role_id	roles.id	ON DELETE CASCADE — ロールを削除するとその割り当ても削除される（ロールの削除自体はまだどの機能も公開していない）
uq_users_normalized_user_name	UNIQUE	users.normalized_user_name	—	ix_users_normalized_user_name で強制
uq_roles_normalized_name	UNIQUE	roles.normalized_name	—	ix_roles_normalized_name で強制

API／DD との対応

テーブル.列	DD の項目	API の項目
users.id	未作成（WI-001 には DD がない。認証のエンドポイントは 0002_ADR で直接定める）	GET /api/auth/me → id
users.user_name	—	GET /api/auth/me → userName、POST /api/auth/login のリクエスト → userName
users.display_name	—	GET /api/auth/me → displayName
roles.name（user_roles 経由）	—	GET /api/auth/me → roles: string[]

マイグレーションの影響と復旧の限界

- **マイグレーションの種類**： 追加のみ — プロジェクト最初のマイグレーションで、保持や変換が必要な既存データはない。
- **データ復旧の限界**： 該当なし — これらのテーブルに既存の行はない。
- **ロールバック計画**： `dotnet ef database update <前のマイグレーションまたは 0>` で、まだ何からも参照されていないこれらのテーブルを問題なく削除できる。この段階ではロールバックは安全。

未決定事項

決定事項	選択肢	推奨	状態
状態を変更するエンドポイントに、 <code>SameSite</code> Cookie 属性に加えて CSRF トークンが必要か	<code>SameSite=Lax</code> のみ / <code>SameSite=Lax</code> + アンチフォージェリトークン	<code>SameSite=Lax</code> のみ — 追加のトークンなし	決定 — <code>work-items/WI-002/decisions.md</code> DEC-020 (2026-09-18)
<code>email_confirmed</code> / <code>two_factor_enabled</code> / <code>phone_number*</code> の列を将来の要件で使うことがあるか	未使用のまま残す / モデルから削る	残す (コストはほぼゼロで、後の互換性のないマイグレーションを避ける) — 決定済み、 <code>users</code> テーブルの備考を参照	決定 — <code>work-items/WI-001/decisions.md</code> (認証基盤の設計) を参照